数据结构 · 链表算法题（下）· 考点精讲与真题整合

408 数据结构大题专项 | 课程第 2 讲（分 P4） | 时长 2 小时 23 分 55 秒

本讲范围：分 P3 那 28 分钟只是本讲的开头，这一段才是「链表算法题」的正片。2 小时 24 分里讲完了编号 17–20 的**四道统考真题**（倒数第 k 个 → 找公共结点 → 删绝对值重复 → 重排链表），题题给完整代码；中间为讲清第 20 题，单独补了三个零件：**求表长、链表反转（头插法）、快慢指针求中点**。

读法建议：§4 是零件库，§5 是装配现场。最值钱的是 §5.4 —— 老师说它是「整本书里代码量最长的一道」，而它的第一、二步直接调用 §4.2、§4.3。§6 的易错点每条都能在他代码里指到具体位置。

目录

1 考情分析：四道真题、分值口径、练习节奏

2 核心概念：头指针 / 结点定义 / 快慢指针模型

3 解题方法论：五个套路

4 典型题型与模板（五个零件）

5 统考真题精解：四道真题全解

6 易错点清单

7 考场速查卡

附 本资料的来源与边界

ASR 校正说明（引用前请先读）

转写稿由本地 Whisper `small` 模型在 CPU 上生成，本讲纯口述代码、没有板书，术语同音错字密度很高。本资料引用时统一校正，主要替换：快慢指针（宽慢指针 / 筷子针 / 快直征）、头结点（投节点 / 投机点）、头指针（投指针）、链表（练表 / 电表）、相交链表（香焦链表 / 枪胶）、字符（制服）、数组（速度 / 宿主）、`true`（触 / 处）、`LinkList`（link 历史 / 另可历史）、`node *`（弄的星）、`for`（佛循环）、`while`（外务循环）、`cur`（COR / COL）、`head`（hide / HAT）、`temp`（Time）、遍历（便利）、负数（复制 / 富数 / 附期）、环（还）、真题（整体 / 灯体）、返回值（反位置）、语法（与法 / 鱼法）。**凡不能确定的，一律不写进正文，集中列在 §8。**

1 考情分析：四道真题、分值口径、练习节奏

本讲在课程中的位置：老师在开场交代了一句实情——分 P3 那一段**忘了点录制**，本讲开头把开头部分重讲了一遍。另需知悉：本次课中间有一次**约 29 分钟的休息**（`[00:48:27]` → `[00:57:31]`），音频里没有内容，**不是转写缺失**。

1.1 老师给的分值口径（本讲最该记住的考情）

老师原话（已校正）	含义
「哪怕你的时间复杂度再高、空间复杂度再高，它这个满分都是 15 分，你只要整个思路正确并且写出来，然后它就是能有个 10 分左右。」（`[01:32:51]`）「但是如果你的思路正确你代码写不出来，那分就是很低。」（`[01:33:05]`）	满分 **15 分**；**思路对 + 代码写出来 ≈ 10 分**，这是保底线；只写思路不写代码 ⇒ 分数很低，**必须落到代码**
「408 里面绝大多数代码一般都 10 行 15 行……但极个别题目可能会代码特别长。」（`[01:28:25]`）「他专门让你写链表的节点定义，这都是有分的……即便这个题目不会写，写一个节点定义都是有分的。」（`[01:21:21]`）	代码量心理预期 10 行量级（第 20 题是那个「极个别」）；**结点定义是独立得分点**，不会做也要先写上去

1.2 四道真题：年份是怎么认定的

> **先看认定依据**
>
> ① **老师亲口说的**：讲末尾他明确说「后面就讲了具体的四道真题」（`[02:21:43]`）—— 所以本节四道题（他编号 17–20）**都是统考真题**；其中只有**第 20 题他说出了年份**：「看一下 19 年这个」（`[01:49:02]`）。
>
> ② **其余三题按题面比对认定（不是老师口径）**：第 17 题 ↔ **2009**「查找倒数第 k 个位置上的结点」（对照点：题面称「带有表头结点的单链表」、只给头指针、要求输出 data 值并返回 1 / 否则返回 0——老师逐句复述了这三条，`[00:04:40]`）；第 18 题 ↔ **2012**「求两个带头结点单链表公共后缀的起始位置」（对照点：`data` 是**字符类型**、要找的是**交点指针而不是值**、题目**没说一定有交点**）；第 19 题 ↔ **2015**「删除绝对值相等的多余元素」（对照点：「**仅保留第一次出现**」+ 题干**专门给出绝对值范围**、辅助数组开 `n+1` 个，与「|data| ≤ n」吻合）。
>
> ③ 题号与年份**请对真题原卷再核一遍**。按年份认定后正好是 2009 → 2012 → 2015 → 2019 **严格递增**，这本身是一条佐证。

老师编号	认定年份	题目	老师本讲有没有说出年份
17	2009	求单链表中倒数第 k 个结点	**没有**（只说「这个题目它让求的倒数第 k 个」）
18	2012	求两个链表的公共结点（第一个交点）	**没有**
19	2015	删除链表中绝对值相等的多余元素	**没有**
20	2019	重排链表 $L' = (a_1, a_n, a_2, a_{n-1}, \ldots)$，要求空间 O(1)	**说了**：「看一下 19 年这个」（`[01:49:02]`）

1.3 老师给的优先级与练习节奏

> 「基本上（哪）题就优先做真题就行了，非真题的话就做我们（课上）画的那几道就可以了。」（`[01:31:22]`）

> 「**一个题目写三四遍比去写三四个题目要强。**」（`[01:29:02]`）「课下一定要写四五遍，不然绝对是记不住的。刚开始你可以两三天写一遍，写一遍之后的话，半个月写一遍……**如果这刚听完的时候你没有写，隔个半月可能就全忘了**，需要重新再去听一下回放。」（`[02:22:00]`）

他给的第一轮标准很宽松，值得照做：「算法题第一遍，你想要把所有的细节都搞明白，很困难……**第一轮还是说把思路搞清楚，你知道（快慢）指针这种用法就可以了。**具体这些语法细节，可能你要再去写个三四遍，可能就真正明白了。」（`[00:11:00]`）

2 核心概念：头指针 / 结点定义 / 快慢指针模型

2.1 头指针与头结点：两个必须分清的概念

> 「头指针永远是指向形式上的第一个结点。所以……**头结点和头指针这两个概念要区分开**：头结点是可以有可以没有，但是（链表题）里面一般都是带头结点的；然后头指针是如果有头结点了就指向头结点，如果没有头结点了就指向实际的第一个结点。」（`[00:07:01]`）

头指针是一个指针变量，永远指向「形式上的第一个结点」——题目里通常叫 `list`（第 17 题）或 `L`（第 20 题），老师平时习惯叫 `head`。**头结点**是链表中真实存在的第一个结点（**不存数据**），可以有也可以没有；链表大题**默认带头结点**（「（真）题一般都是带头结点的」`[01:50:47]`），算表长时**不计入长度**。

由此推出两条硬规则：① 算长度、找第一个实际结点，一律从 `head->next` 起；② 但只要涉及「走一步 / 走两步」的移动次数，**头结点是要算一步的**——这就是 §2.4「长度 7 却要走 8 步」的由来。

2.2 结点定义：必须「和题目保持一致」

四道真题的题干给出的结点定义**各不相同**，老师每道题都现场重写了一遍：

> 「一定要和题目保持一致，这里叫 data 和 link，你不要就不要写成这个 `next`，一定要保持一致。」（`[00:03:19]`）「这些细节也都要对（上）。因为有时候（你）习惯写上 `next`，（那）可能就有（人）说：我题干就对应不上。」（`[00:12:23]`）

> 「它（第 20 题）把节点定义给你写好了……注意审题。像我们之前写这个节点都是 LNode 的，它现在给你定义好了，你就得用（它给的）Node；不要（写）得和题干这个对不上。」（`[01:50:15]`）

题目	题干给的指针域名	数据域类型	老师当场写下的定义（校正后）
第 17 题	`link`	`int`	`typedef struct LNode { int data; struct LNode *link; } LNode, *LinkList;`
第 18 题	`next`	**`char`**	同上，把 `link` 换 `next`、`int` 换 `char`——「换一下类型就可以」
第 19 题	`link`	`int`	「他也是用的和题目保持一致……所以我们这里也用 link」
第 20 题	`next`	`int`	**题干直接给了。**「你就用 `node *` 去代替就可以了」（拼写以原题为准，见 §8）

第 18 题的「变通」提醒：「这个 `data` 应该定义成**字符类型**……所以一定要去**灵活变通**。」（`[00:27:43]`）结点定义不是背模板抄，而是**照着题干给的域名和类型抄一遍**。
顺带一条语法规范：「如果你想一行写两个指针变量，那就只能让这个星号贴近变量名」——`LNode *fast, *slow;` ✓；写成 `LNode* fast, slow;` 就只有 `fast` 是指针。（`[00:08:31]`）

2.3 快慢指针的工作模型：为什么「长度 7 能走 8 步」

> 「这个链表的长度虽然只有 7，但是如果你**从头结点开始往后走，其实能走 8 步**。就是如果快指针走到空的时候，慢指针始终还在这个起点，那就说明是不存在的。」（`[00:05:38]`）

带头结点的单链表，7 个数据结点（1…7）：

```
list → [头] → [1] → [2] → [3] → [4] → [5] → [6] → [7] → NULL
         ↑      ↑                                      ↑
       第 0 步  第 1 步                              第 8 步 = NULL

从头结点到 NULL 一共 8 步 —— 双指针移动的「圈数」按 8 算，
而「倒数第 k 个」里的 k 按 7 算。k ≥ 8 时快指针先到 NULL，慢指针一步没走。
```

把这张图浓缩成一句话：慢指针「一步没动」就是「不存在」的判定条件。所以本讲四道题反复出现同一种收尾：循环退出后先看慢指针还在不在起点。第 17 题写成了 `if (slow == list) return 0;`（老师当场写错过一次，见 §5.1.3）。

2.4 求链表长度：老师说是「极其高频」的代码

> 「**链表求长度的代码要会写，这种也是极其高频**……但是**不要硬背，在理解的基础上去记**。」「这两种写法你用哪一种都可以，看你到时候哪个好理解好记一点就写哪一个。」（`[01:31:34]`、`[00:37:20]`）

两种写法的代码见 §4.1。

3 解题方法论：五个套路

3.1 套路一：双指针 —— 本讲的核心思想

> 「总体上**双指针这个思想特别重要**，从历年（真）题（来看），很多题目都是这个思想相关的，一定要总结起来。」（`[02:21:33]`）

用法	推进规则	用于哪道题
固定间距	快指针先走 k 步，之后两个一起走	第 17 题「倒数第 k 个」：快指针到 NULL 时慢指针正好停在答案上
对齐起点	先把长的那条链表的指针往前挪「长度差」步，再一起走	第 18 题「找公共结点」：两指针到链尾距离相等后，必然同时到达交点或同时到 NULL
速度 2 : 1	快指针每次 2 步、慢指针每次 1 步	第 20 题「找中点」；课后题「判断是否有环」（有环必相遇）

3.2 套路二：链表删除 —— 指针一定要停在前一个结点

> 「**链表的删除一定是这样的：你要删除这个节点，指针一定要停在前面**，你只有停在前面，你才能跳过它……如果你要删除的就是（当前指针指的）这个，没法删的，因为他没法去改前面这个指针了。」（`[01:17:36]`）

```
删除 p 后面的那个结点（目标结点叫 q = p->next）：

   … → [p] → [q] → [r] → …          删除前
   … → [p] ————————→ [r] → …        删除后：p->next = q->next;（= p->next->next）

   ★ p 自己不动！删完之后 p 后面自动换成了 r，下一轮接着判 r；
     这时如果还让 p 后移，就会漏判 r。
```

因此第 19 题的起点是「头结点」，不是第一个数据结点：「你最开始一定得让指针指向开头，用来判断后面要不要删。等到指针指向 21 的时候，你得去判断负数要不要删，**不是判断当前这个位置**。」（`[01:18:21]`）`cur` 从头结点出发，判的永远是 `cur->link` 指向的那个结点——这样第一个数据结点也才在删除范围内（它的前驱是头结点，改得动）。

3.3 套路三：链表反转 —— 头插法

> 「在做二十题之前，我们要先讲一个链表反转，这也是一个很经典的问题……**如果这一块你不熟，看那个题目会相当困难。**」（`[01:30:33]`）

五步固定顺序：① `head->next = NULL;`（最容易漏）→ ② `temp = cur->next;`（存后继，不存就断链）→ ③ `cur->next = head->next;` → ④ `head->next = cur;` → ⑤ `cur = temp;`（不推进就死循环）。完整代码见 §4.2。

为什么第 ① 步可以放心断开：「**因为你已经把这个 1 号结点放在 cur 了，后面就不用担心找不到。**就算你把 head 后面置为空，通过 cur 照样能定位到这个 1。」（`[01:38:40]`）—— 这就是「先把要用的结点拿在手

里，再动指针」。完整代码见 §4.2。

3.4 套路四：数组兜底法（本讲升级版）

> 「**数组很万能**，在链表（题）的时候很万能。因为**链表不支持随机访问**，只能说一个一个去遍历，但是你只要把它放到数组就很好操作。这种做法就是考试实在没有其他办法的话可以用一用。」（`[01:13:29]`）

三个档次，按开销从小到大：① 借数组随机访问（第 17 题：求长度 → 存数组 → 取 `A[n-k+1]`）；② **数组 + 辅助标记**（第 19 题：再开一个数组当「出现过没有」的表，在原数组上删减）；③ **数组 + 重建链表**（反转、第 20 题重排：倒着按数组建一条新链表）。三者都是「思路特别好想也特别好写」。另外老师提醒：「（第 20 题）让空间复杂度 O(1)，不用管……**随便写都比（空着）强**。」（`[01:51:38]`）

老师反复强调的备考观（本讲说了三遍）：「一定要学算法题，**一定要把思路打开，就是同一个题目多想想其他做法**。」（`[01:53:32]`）「**千万不要仅限于答案的那一种**。如果你只会一种做法，考试可能换个题目就不会做了。」（`[01:53:46]`）「**你一定要注意代码的易读性**。」（`[00:44:51]`）

3.5 套路五：答题格式与注释策略

答题四件套：**结点定义 → 设计思想 → 代码（10～15 行量级）→ 时空复杂度**——「**第一问是写思路，（第）三问分析时空复杂度**」（`[00:17:20]`）。注释只按逻辑段写：「考试你写注释不需要每一句都写，**整个这个 while 循环你写一下「反转链表」就够了**。」（`[02:03:22]`）

4 典型题型与模板（五个零件）

这五个模板都是老师为讲第 20 题而单独讲的「零件」。第 20 题的完整代码，本质上就是**模板 C（找中点）+ 模板 B（反转）+ 模板 D（交替合并）**拼起来的。

4.1 模板 A：求表长（两种写法）

```
/* 写法一：cur 从第一个实际结点出发 */
int Length(LinkList list) {
    int x = 0;
    LNode *cur = list->next;      /* ★ 头结点不算长度，从头结点后面那个结点开始 */
    while (cur != NULL) {
        x++;                      /* 当前结点不是空，长度加一 */
        cur = cur->next;          /* 指针往后走一步 */
    }
    return x;
}
/* 写法二（老师：「就是王道书上的写法」）：只把初值改成 cur = list，
   循环条件改成 while (cur->next != NULL)，其余一样 —— x++ 与 cur = cur->next; 不变 */
int Length(LinkList list) {
    int x = 0;
    LNode *cur = list;            /* ★ 和写法一就差在这个初值 */
    while (cur->next != NULL) {   /* 后面还有结点才进循环 */
        x++;
        cur = cur->next;
    }
    return x;
}
```

老师逐句念了写法一的执行过程（长度 3 的链表，x 从 0 走到 3、遇到 NULL 退出）：「退出的时候刚好这个变量刚好就是三。」（`[00:34:00]`）

4.2 模板 B：链表反转（头插法，空间 O(1)）

```
/* 就地反转带头结点的单链表，空间 O(1) */
void Reverse(LinkList head) {
    LNode *cur = head->next;      /* cur 指向第一个（实际）结点 */
    head->next = NULL;            /* ★ 最容易漏的一句 */
    while (cur != NULL) {
        LNode *temp = cur->next;  /* ① 先存后继 */
        cur->next = head->next;   /* ② 把 head 后面那一整串接到 cur 后面 */
        head->next = cur;         /* ③ 再把 cur 接回 head 后面 */
        cur = temp;               /* ④ cur 往后走 */
    }
}
```

4.3 模板 C：快慢指针找中点

```
LNode *slow = head, *fast = head;              /* 都从头结点出发 */
while (fast != NULL && fast->next != NULL) {  /* ★ 两个条件都不能少 */
    fast = fast->next->next;
    slow = slow->next;
}
/* 奇数个：slow 停在正中间；偶数个：slow 停在前半段的最后一个 */
```

> 「**这个要分两种情况：一种是奇数个结点，还有一种是偶数个结点。**……快指针走到头的时候，慢指针刚好停在正中间，这是奇数个（奇偶都**不包含头结点**）。」（`[01:46:57]`）「（偶数个时）快指针走不了两步了，只能再走一步，那就得停了……**这时候其实慢指针停留的位置应该是前半段的最后一个。**」（`[01:48:30]`）

循环条件为什么必须写两个（老师在第 20 题处专门讲）：「首先**快指针本身不能等于空**，然后**快指针的后面也不能等于空。**因为快指针走两步，如果是空的话你是不能再往后走的……所以要专门加上这个判断语句。」（`[01:58:09]`）

4.4 模板 D：两条链表交替合并（轮流取一个）

```
LNode *p = A->next;              /* 前半段的第一个实际结点 */
LNode *cur = A;                  /* ★ cur 充当尾指针，指向合并后的最后一个结点 */
while (q != NULL) {              /* ★ 一定以短的那条（后半段）为准 */
    cur->next = p;  p = p->next;  cur = cur->next;
    cur->next = q;  q = q->next;  cur = cur->next;
}
cur->next = p;                   /* ★ 奇数个时前半段还剩一个；偶数个时接的是 NULL，无害 */
```

> 「cur 也得往后走，cur 充当一个尾指针的作用……**最外面还要再加一句这样一句话，不能漏了。**」（`[02:08:53]`）

4.5 模板 E：`map` 标记「这个结点走过没有」

```
#include <map>
map<LNode *, bool> dic;    /* ★ 键必须是指针：存的是「结点地址」，不是结点里的值 */
dic[cur] = true;           /* 走过就标记（老师口述为「置为 true」）；再次遇到 if (dic[cur]) 即成 */
```

他点名的「常见的坑」：「**你一定要定义成 `LNode *`。**有的人可能想着去定义成字符，这是不允许的……因为很有可能出现**相同的字符**，但是他们这个指针不一样，**在内存上的这个地址是不一样的**，所以他依然不是同一个结点。」（`[01:04:59]`）

5 统考真题精解：四道真题全解

5.1 2009 第 17 题 · 求倒数第 k 个结点（15 分）

5.1.1 题面要点与三种方法

> 已知一个带有表头结点的单链表，结点结构为 `| data | link |`，只给出头指针 `list`。在不改变链表的前提下设计一个尽可能高效的算法，查找倒数第 k 个位置上的结点（k 为正整数）。若查找成功，输出该结点的 `data` 值并返回 1；否则只返回 0。（题面引自公开真题资料，**完整题面请对原卷**）

> 「有返回值的，返回 1 或者返回 0，那相当返回是一个 `int`。」（`[00:04:19]`）「如果链表只有三个节点，现在让你去求倒数第四个，那肯定没有……**如果你的倒数第 k 个位置超过了这个链表的长度，那肯定是不存在。**」（`[00:04:40]`）

> 「第一种方法就是说你转成数组去做；第二种方法就是说先求长度，第一趟先统计一下链表的长度，第二趟再去求这个倒数第 k 个。**然后前两种方法我们都不去写代码了……我们就重点讲第三种叫快慢指针。**」（`[00:01:58]`）

三种方法：① 转成数组——求长度 → 开数组存 → 直接取 `A[n-k+1]`（「你要开一个 n 大小的数组，那空间复杂度就 O(n)；你想找到倒数第 k 个那就 O(1)，因为数组访问任何一个元素都是 O(1)」）；② **两趟扫描**——第一趟求长度 n，第二趟走到正数第 n−k+1 个（老师未展开）；③ **快慢指针**——快指针先走 k 步，再两个一起走，快指针到 NULL 时慢指针即答案（「时间是 O(n)，这个链表总共扫描一遍；空间是 O(1)」）。

⚠️ **方法① 的复杂度：转写这一句不干净：**原转写是「所以如果用（数组）那种方法是 **O1 和 O1**」（`[00:19:13]`）。按上下文（开 n 大小的数组、把链表存进数组需 O(n)、之后按**下标**取是 O(1)）推断，他说的应是**空间 O(n)、查表 O(1)**；「O1 和 O1」很可能是「O(n) 和 O(1)」的听写错误。**本资料按 O(n)/O(1) 记录并在此标出。**

5.1.2 结点定义（题干叫 `link`，就必须写 `link`）

```
typedef struct LNode {           /* 题干给的域名叫 data 和 link，照抄 */
    int data;
    struct LNode *link;          /* ★ 不能写成 next */
} LNode, *LinkList;

int func(LinkList list, int k);  /* 签名：返回 int；参数是一条链表 + 一个 k */
```

5.1.3 写法一：先走 k 步 + 一起走

```
int func(LinkList list, int k) {
    LNode *fast = list;                    /* 快指针指向头结点 */
    LNode *slow = list;                    /* 慢指针也等于这个头指针 */

    /* ① 快指针先走 k 步 —— 必须带「快指针不为空」的判断 */
    while (k > 0 && fast != NULL) {        /* ★ 少写 fast != NULL 是绝大多数人的第一反应，也是错的 */
        fast = fast->link;
        k--;
    }

    /* ② 快指针走完 k 步后，快慢指针一块走 */
    while (fast != NULL) {
        fast = fast->link;
        slow = slow->link;
    }

    /* ③ 慢指针一步都没动 ⇒ 倒数第 k 个不存在 */
    if (slow == list) return 0;            /* ★ 老师当场把这一句写错过，见下 */
    printf("%d\n", slow->data);            /* 输出并返回 1 —— 对照题干的「否则」要求 */
    return 1;
}
```

关键点 1：那个「与运算」。「这个绝大多数人都会这样去写，**这个写法是有问题的。因为你这个 k 很有可能超过（链）表的长度**——比方说这个链表的长度才只有 7，你让它走 10 步，肯定走不了的。**所以这里一定要加**

上判断条件，这个是很容易刚学（的人）被忽略的。……一旦等于空了就得停了，**哪怕还没有走 k 步也得停。**」（`[00:09:28]`）

关键点 2：慢指针没动就是「不存在」。「如果你要找倒数第 3 个，让快指针先走三步，之后如果两个同时走的话，快指针走到头，慢指针会刚好停留在倒数第三个位置，**因为它永远落后于这个快指针三步。**」（`[00:13:10]`）

> ⚠ **老师当场写错并当场改了这一句**
>
> 「这块代码的话，**我们这里写的应该是写错了，这里应该是「等于」。**如果慢指针没有（动），那才是返回零。」（`[00:14:43]`）
> —— 判断「慢指针还在起点」要用 `==`，不是 `!=`。写成 `if (slow != list) return 0;` 逻辑正好反了（会在找到答案时返回 0）。

5.1.4 写法二：把两个循环合并（精简版）

「就这里的这几句其实可以合到一块……你想假设说你要找到倒数第 k 个节点、k 等于 3，那可以（先）减减——这个（用）编程语言就**先用再减**：k 等于 3 的时候并不满足小于等于 0，k 等于 2 的时候也不满足，k 等于 1 的时候也不满足；**在前三次慢指针不走，快指针去走；等到第四次这个 k 等于 0 了，这时候快慢指针都会走。**」（`[00:20:54]`）

```
int func(LinkList list, int k) {
    LNode *fast = list, *slow = list;       /* 一行两个指针：星号贴近变量名 */
    while (fast != NULL) {                  /* 一个循环搞定 */
        fast = fast->link;
        if (k-- <= 0) {                     /* ★ 后置减：先拿 k 的当前值比较，再减 1 */
            slow = slow->link;              /* 前 k 次慢指针不走 */
        }
    }
    if (slow == list) return 0;             /* 后半段和写法一一模一样 */
    printf("%d\n", slow->data);
    return 1;
}
```

以 k = 3 为例：前三轮判断都是「否」（k 依次变 2、1、0），只有 `fast` 走；第四轮 `0 <= 0` 成立，此后恒成立，**两个指针同步走 ⇒ slow 永远落后 3 步 ✓**

「（两种写法）时间复杂度是一样的，只不过左边是把快指针在走的过程分成了两部分，右边都写在一块了，并不会影响时间复杂度，代码的写法上更优化了。」（`[00:21:24]`）

⚠ **写法二的确切形式是推断的：**老师这一段是口述（「先用再减」「并不满足小于等于 0」），**没有逐字念出循环体。**本资料还原为 `if (k-- <= 0) slow = slow->link;`，依据是：①「先用再减」= 后置减，说明 `k--` 写在判断里；②「不满足**小于等于 0**」直接对应判断式 `k <= 0`；③ 四次 k 值 3/2/1/0 的走向与他的描述逐条吻合。**写成 `if (k > 0) k--; else slow = slow->link;` 完全等价**，考场任选一种。

5.1.5 复杂度

「时间复杂度就是 O(n)（快指针从头走到尾）……空间复杂度是因为你没有去开额外的空间，所以是 O(1)。」（`[00:17:54]`）

5.2 2012 第 18 题 · 求两个链表的公共结点

5.2.1 题面要点与题眼

> 「它也是快慢指针的应用。比如说第一条链表它的长度是 7，第二条链表的长度是 5，那你让长度长的那条先走两步，慢的先不走，之后让两个同时去走，然后他们就会在这个交点相遇。」（ [00:26:47] ）

> 「所以你要找出交点的这个**指针**，**不要把这个（值）输出**……最终你要找的并不是这个（结点）里面的值，而是这个指针。」（ [00:27:20] ）「这个 `data` 应该定义成**字符类型**……然后这里的话名字叫 `next`。」（ [00:27:39] ）

题眼是「没有交点怎么办」：

> **「无论两个链表相不相交，你退出循环的时候让两者相等就行了。你的循环退出条件是两个指针相等……因为题目里并没有说一定存在交点，所以你的代码要能处理没有交点的情况。」**（ [00:28:02] ）

5.2.2 写法一：长度差 + 同步走

```
/* 求两条带头结点单链表 str1、str2 的第一个公共结点；无公共结点时返回 NULL */
LNode *FindCommon(LinkList str1, LinkList str2) {
    int len1 = Length(str1);            /* ★ 求表长的代码要自己实现（见 §4.1） */
    int len2 = Length(str2);

    LNode *p = str1, *q = str2;         /* 都从头结点出发；★ 不要用 fast/slow 命名，见下 */

    /* 让长的那条先走「长度差」步 */
    if (len1 > len2) {                  /* ★ 一定是「大于」，不能写「大于等于」 */
        while (len1 > len2) { p = p->next; len1--; }
    } else {
        while (len2 > len1) { q = q->next; len2--; }
    }

    /* 两个一起走，直到相等 */
    while (p != q) { p = p->next; q = q->next; }
    return p;                           /* 有交点返回交点；无交点时两者同时为 NULL ✓ */
}
```

> **两个写法上的讲究（都是老师原话）**
>
> **① 别用 fast/slow 命名：**「**尽量不要去用这个 fast 和 slow**。因为两条到底谁长谁短你提前不知道……**这最好就定义成 p、q，更规范点**；你定义成快慢的话，它并不能提前确定谁快谁慢。」（ [00:40:52] ）
>
> **② 这个 if/else 结构值得学：**「因为你不知道哪个链表的长度更长，所以谁先走你也是不确定的。注意学习下这种写法，**包括答案也是这样写的**……这种写法是特别简化的。」（ [00:41:16] ）

关于「大于 / 大于等于」，老师用具体例子讲了怎么自查：

> 「**你一定要把这个（写成）大于，不要写成大于等于。**你考试的话都可以通过具体的例子去试一下：比如说一个长度是 7、一个长度是 5，你肯定让更长的那个先走两步；那你如果（写成）**大于等于（len2），那相当于 7 的时候会**

> **走，6 的时候会走，5 的时候还会走，那相当于走 3 步了。**这种通过一个例子就很明显地能判断出来。」（`[00:43:01]`）

> ⚠ **这两个循环写成 `while` 还是 `for`，转写看不出**
>
> 老师说完「大于 / 大于等于」后紧接着说「所以这两个 `for` 循环也会（只）执行一个」（`[00:42:54]`），说明他心里想的可能是 `for`。**本资料按 `while (len1 > len2) { p = p->next; len1--; }` 还原**，理由是只有这种「一边比较一边把 `len1` 减下去」的形式，他举的「7 会走、6 会走、5 还会走（共 3 步）」才精确成立。他另提过一种等价写法：「求一个 `x` 等于（两者相减的）绝对值……第一个大你就让这个 p 先走这个 x 步……这样写也是没问题的。」（`[00:43:47]`）

5.2.3 写法一的复杂度

> 「**你求两条链表的长度也要算进时空复杂度里面**……第一条是 m、第二条是 n，求长度要把两条都走一遍，所以是 m+n；第二次他们只会走到焦点，一定是小于（m+n）……**就分析个时间复杂度是 m+n 就行了，主要是求长度的时候占用的开销。**」（`[00:45:44]`）

5.2.4 写法二：用 `map`（老师说是书上没有的写法）

> 「你通过一个 map 先把第一条链表只要走过就把它……走过之后你把它的地址置为 `true`。你先把第一条链表扫描完，这第一条链表的所有节点都被置为 `true` 了；那你再去遍历第二条链表，**第一次碰到这个 `true` 的那就是交点。**」（`[01:01:16]`）

```
/* 写法二：用 map 记录「第一条链表里出现过哪些结点地址」 */
LNode *FindCommon2(LinkList str1, LinkList str2) {
    map<LNode *, bool> dic;         /* ★ 键必须是指针，不能是 char —— 见 §4.5 */
    LNode *cur = str1->next;        /* 从头结点后面那个结点开始 */
    while (cur != NULL) {
        dic[cur] = true;            /* 走过就置为 true */
        cur = cur->next;
    }
    cur = str2->next;               /* ★ cur 已经定义好了，直接二次利用，不用再定义变量 */
    while (cur != NULL) {
        if (dic[cur]) return cur;   /* 第一次碰到 true 的就是交点 */
        cur = cur->next;
    }
    return NULL;                    /* 走完都没找到 ⇒ 没有交点 */
}
```

「（这种写法）空间会大点……但是说他这个做法很容易想，基本没有什么思考的量，而且写起来代码特别短。」（`[01:02:24]`）「**空间是 O(n)**，因为你用了一个 map，就相当于用了一个辅助数组，只不过这个辅助数组的下标可以取任意值；**然后时间也是 O(n)。**」（`[01:06:29]`）

顺带布置的课后题（非真题，他题单上的第 15 题）：判断链表是否有环。方法①**用 map**：「往前走的过程中发现一个之前是 `true` 的，那就说明有环」；方法②**快慢指针**：「一个指针每次走两步，另外一个每次走一步，**它们一定会（在）环（里）相遇的**；没有环就永远不会相遇。」（`[01:07:24]`）

5.3 2015 第 19 题 · 删除绝对值相等的多余元素

5.3.1 题意与题目给的条件

> 「他说**时间上尽可能高效，那就空间其实无所谓**……然后他说**仅保留第一次出现**（绝对值相等的结点）。」（`[01:12:49]`）

> 「这个辅助数组的话，**题干上已经告诉你了他的（绝对值的范围）**……因为你要用辅助数组必须知道他的这个绝对值多大。这个题干上专门告诉你他是（小于 n 的），所以你辅助数组要开（`n+1`）个就可以了。」（`[01:16:56]`）

⚠ **这一句转写不干净，请对题目核：**原转写是「这个提供上专门告诉你他是**小元的**」（`[01:17:24]`）。「小元的」无法直译，按下文「辅助数组要开 `n+1` 个」推断为「**小于 n 的**」。这一条恰好就是该真题题干里的「|data| ≤ n」条件，但**老师本讲的原句不敢逐字引用**。

老师举的例子（⚠ 原始序列请看题目图）：「21 绝对值就第一次出现保留，（负 15）第一次出现保留，这个（负 15 再出现）之前出现过了就不要；这个（负 7）第一次出现保留，这个 15 的话绝对是之前出现过就不要。」（`[01:13:08]`）最后——「这个链表就变成了 **21 - 15 - 7**。」（`[01:27:56]`）→ **可确证的是结果 `21, -15, -7`**。

5.3.2 链表写法（在原链表上直接删，不新建结点）

> 「**这个题目他没有要求返回值，所以你就可以直接不要返回值，直接在原链表上操作就行。**（传）进来的参数只有一条链表。」（`[01:19:15]`）

```
void DeleteDup(LinkList head, int n) {
    int *a = new int[n + 1];              /* ★ 题干给出 |data| ≤ n，最大取到 n，所以要开 n+1 个 */
    for (int i = 0; i <= n; i++) a[i] = 0; /* ← 讲稿没有念这一句，见下方警告框 */

    LNode *cur = head;                    /* ★ cur 从头结点出发：删除必须停在「前一个结点」上 */
    while (cur->link != NULL) {           /* cur 后面还有结点才需要判断 */
        int v = cur->link->data;          /* 取出后面那个结点的数值 */
        if (v < 0) v = -v;                /* 求绝对值（讲稿没念是 abs() 还是手写 if） */

        if (a[v] != 0) {                  /* 之前出现过 ⇒ 删掉它 */
            cur->link = cur->link->link;  /* ★ cur 不动！下一轮继续判新的 cur->link */
        } else {
            a[v] = 1;                     /* 第一次出现 ⇒ 记下来（老师：「也可以写 a[v]++ 统计次数」）
*/
            cur = cur->link;              /* ★ 只有「没删」的时候 cur 才后移 */
        }
    }
}
```

老师把这段代码**对着图模拟执行了一遍**，关键两处：①「因为你刚开始创建完这个数组，它里面什么都没有存，**所以肯定它是等于零的**」；②「那 `cur` 就会把后面那个节点删掉，直接跳过它指向下一个，**但是 `cur` 没动、还在这**，再进行下一次循环。」（`[01:25:51]`、`[01:26:47]`）

⚠ **辅助数组的初始化：老师假定它是 0，讲稿没有念那一句**

老师说「它里面什么都没有存，所以肯定它是等于零的」——他是**假定**辅助数组初值为 0 的。但 C/C++ 里 `new int[n+1]`（不带括号）**不会**清零，本资料因此在代码里补了一行显式清零并标出（考场稳妥写法：`new int[n+1]()`）。**这一行不是老师念的。**

这道题最容易写错的一处，老师点了两遍

删掉一个结点之后，`cur` 不能后移。删完之后 `cur->link` 已自动变成原来后面的那个结点，再让 `cur` 往后走就会**漏判**它。整段节奏是：「删」分支里只有 `cur->link = cur->link->link;`，`cur` 不动；「留」分支里才 `cur = cur->link;`。

5.3.3 复杂度与备选解法

「整个这个题目的时间复杂度 O(n)，把整个链表扫描了一遍；空间复杂度，因为用了一个辅助数组，也是 O(n)。」（`[01:28:04]`）

「（数组法）你开了两个数组，而且又新建了一条链表，空间很大，**但是说这种思路特别好想也特别好写，这种题目特别万能**。当然这种思路我们就提一下就行了，就不具体去实现了。」（`[01:13:29]`）

5.4 2019 第 20 题 · 重排链表（本讲压轴，代码量最大）

5.4.1 题面与三步思路

「最后一道真（题）……看一下 19 年这个。」（`[01:48:54]`）「它最后想让你先输出第一个，先第一个再最后一个，第二个再倒数第二个……你看 172635 还缺一个 4。」（`[01:51:05]`、`[02:15:37]`）

即把 L = $(a_1, \ldots, a_n)$ 重排成 L′ = $(a_1, a_n, a_2, a_{n-1}, \ldots)$，要求空间 O(1)。老师的图用 `1 2 3 4 5 6 7` 举例，结果是 `1 7 2 6 3 5 4`。

「注意审题……它把节点定义给你写好了，你就得用它给的 Node，不要（写）得和题干这个对不上。然后它也没有重新定一个专门（叫）LinkList 的（别名），所以**你就用 `node *` 去代替就可以了**。」（`[01:50:23]`）

「就是思路很简单：**你先找到终点**；**找到终点之后，你把后半段反转**；**反转完之后，你前面取一个、下面取一个、上面取一个、下面取一个**……所以就三步：第一步找到终点，第二步后半段反转，第三步（合并）。思路是很好想，但是这个写代码会（很繁琐），代码量还是挺大的。」（`[01:54:28]`）

```
步 1   快慢指针找中点                          用 §4.3 模板 C
       循环条件必须写 fast != NULL && fast->next != NULL
步 2   把 slow 当「头结点」，反转它后面那半段   用 §4.2 模板 B（head 换成 slow）
       「这个 slow 就相当于刚才反转链表的这个 head，这里代码一模一样」
步 2.5 把两条链表拆开
       ★ 必须先 q = slow->next; 再 slow->next = NULL;，顺序反了就找不到后半段
步 3   两条链表交替合并                        用 §4.4 模板 D
       以短的那条为循环条件；退出后还要补一句把剩下的 p 接上
```

5.4.2 完整代码

```
/* 重排带头结点单链表 L：L' = (a1, an, a2, an-1, ...)，空间 O(1)
   老师说「起一个函数叫 FUNC」—— 函数名并没有真的给出，这里是整理时所加 */
void func(node *L) {

    /* ===== 第一步：快慢指针找中点 ===== */
    node *fast = L, *slow = L;
    while (fast != NULL && fast->next != NULL) {  /* ★ 两个条件都不能少 */
        fast = fast->next->next;
        slow = slow->next;
    }
    /* 奇数个：slow 停在正中间；偶数个：slow 停在前半段的最后一个 */

    /* ===== 第二步：反转后半段（slow 当「头结点」，它自己不参与反转）===== */
    node *cur = slow->next;       /* cur 指向后半段的第一个 */
    slow->next = NULL;            /* ★ 同 §4.2 的那一句，这里是为反转做初始化 */
    while (cur != NULL) {         /* 这个 while 就是「反转链表」，注释写到这一行就够了 */
        node *temp = cur->next;
        cur->next = slow->next;
        slow->next = cur;
        cur = temp;
    }
    /* 反转完：本来 4 后面是 5-6-7，现在变成 7-6-5，但两半段还连在一起 */
```

```
    /* ===== 第三步：拆成两条链表 ===== */
    node *q = slow->next;         /* ★ 先把后半段取出来…… */
    slow->next = NULL;            /* ★ ……再断开。顺序反了就找不到 q 了 */
    node *p = L->next;            /* p 指向前半段的第一个 */
    cur = L;                      /* cur 二次利用，充当尾指针 */

    /* ===== 第四步：轮流取，以后半段（短的）为循环条件 ===== */
    while (q != NULL) {
        cur->next = p;  p = p->next;  cur = cur->next;   /* 上面取一个 */
        cur->next = q;  q = q->next;  cur = cur->next;   /* 下面取一个 */
    }
    cur->next = p;                /* ★ 奇数个时前半段还剩一个；偶数个时 p 为 NULL，接上去无害 */
}
```

5.4.3 三个不得不注意的顺序问题

① 拆链时「先取后断」。「这也是大多数人刚学的时候、刚看这个题很容易忽略的一句。」（`[02:03:03]`）「你如果把（`slow`）后面置为空，这时候就找不到 q 了……**所以一定要先把它取出来**，先把后半段的取好之后，然后再来把这个断开。」（`[02:06:21]`）

```
正确：  node *q = slow->next;   slow->next = NULL;       /* 先取，后断 ✓ */
错误：  slow->next = NULL;      node *q = slow->next;    /* 断完再取，q 恒为 NULL ✗ */
```

② 循环条件用 `q`。「如果整个链表是奇数个，上面会多一个。**所以你这个循环判断条件一定要以下面这个为主**，因为下面走到空的时候，其实上面还有最后一个。」（`[02:05:14]`）

③ 退出循环后必须补一句。「退出循环你这个链表还差一个……**这个代码最外面还要再加一句这样一句话，不能漏了。**」（`[02:11:39]`）「如果两条链表长度相等，那 q 等于空的时候 p 其实也等于空……多加一个空其实无所谓。」（`[02:12:16]`）

合并过程的逐步模拟（老师口述版，用 `1 2 3 4` | `7 6 5`）：

```
轮1   接上 p=1（p→2）   接上 q=7（q→6）    →  1, 7
轮2   接上 p=2（p→3）   接上 q=6（q→5）    →  1, 7, 2, 6
轮3   接上 p=3（p→4）   接上 q=5（q=NULL）→  1, 7, 2, 6, 3, 5，退出循环
补    cur->next = p;  把剩下的 4 接上   →  1, 7, 2, 6, 3, 5, 4  ✓
```

5.4.4 复杂度、老师的评价与同题改编

> 「这个链表相当遍历了两遍……但是因为时间复杂度是可以忽略常数项的，所以就不用管；然后空间复杂度就 O(1)。」（`[02:13:15]`）

> 「这个题目属于思路都很简单，但是写起来会很繁琐。」（`[02:10:15]`）「代码量应该是（整本书）里面最长的一个……如果扣掉这个反转链表这一部分，应该就会没有那么长了。」（`[01:54:19]`）

当堂布置的作业 —— 判断回文链表：「方法和上面那个题一模一样：先找到终点，然后把后半段反转，反转完之后一个一个去对比就行了。」（`[02:18:05]`）三处差别：① 后半段反转之后**不合并**，「只去判断 p 和 q 的值是否相等就行」；② 遇到不相等要**提前退出循环**——「第一次出现不相等就可以停了」；③ 最后补 `p` 的那一句**不需要**——「如果是奇数个，你就算多那个节点不影响回文链表的（判断）。」

6 易错点清单

下表每条都出自本讲老师的原话或他当场踩的坑，最后一列给出时间戳以便回听核对。

易错的写法 / 想法	为什么错	正确做法	出处
快指针先走 k 步时不加「不为空」的判断；判「慢指针是否还在起点」时用 `!=`	前者：「k 很有可能超过（链）表的长度」，会越界；后者：逻辑正好反了，会在找到答案时返回 0（**老师当场写错、当场改**）	`while (k > 0 && fast != NULL)` ——一旦为空就停；判断用 `if (slow == list) return 0;`	`[00:09:35]` `[00:14:43]`
结点定义照抄模板：域名写成 `next`、类型一律写 `int`	「题干就对应不上」，无谓失分；第 18 题题干要求的是**字符类型**	「一定要和题目保持一致」「一定要去灵活变通」——域名与类型都跟着题干走	`[00:12:12]` `[00:27:39]`

易错的写法 / 想法	为什么错	正确做法	出处
删除时从第一个数据结点起步，或删完让 `cur` 也后移	前者「没法去改前面这个指针了」；后者 `cur->link` 已换成下一个结点，再后移就漏判它	「最开始一定得让指针指向开头（头结点）」；「删」分支里 `cur` **不动**，只有「留」分支才 `cur = cur->link;`	`[01:18:07]` `[01:26:47]`
反转时漏掉 `head->next = NULL;`，或头插法里不先用 `temp` 存后继	前者「这句很容易刚学的话（被）漏掉」，没有反转的终止位置；后者改完 `cur->next` 就再也找不到原来的后一个结点了	五步固定顺序：先断头结点 → 存后继 → 接上已反转串 → 挂回 `head` → `cur = temp`（完整代码见 §4.2）	`[01:37:53]` `[01:38:57]`
快慢指针求中点只写 `fast != NULL`，或不区分奇偶	循环体要用 `fast->next->next`，`fast->next` 为空时越界；奇数个停**正中间**、偶数个停**前半段最后一个**	`while (fast != NULL && fast->next != NULL)`；合并时一定以**短的**那半段为循环条件	`[01:58:09]` `[01:46:57]`
拆链时先写 `slow->next = NULL;` 再取 `q`；一行写 `LNode* fast, slow;`	前者「你一旦把它置为空就找不到了」，后半段整个丢失；后者只有 `fast` 是指针	拆链**先取后断**；一行两个指针时「只能让这个星号贴近变量名」	`[02:06:21]` `[00:08:31]`
合并循环结束后不补 `cur->next = p;`	奇数个结点时「退出循环你这个链表还差一个」，会漏掉中间那个	「最外面还要再加一句这样一句话，不能漏了」（偶数个时接的是 NULL，无害）	`[02:11:39]`
用值类型当 `map` 的键 / 辅助数组忘开 `n+1` 个或忘清零 / 第 18 题把指针命名成 `fast`、长度差写成「大于等于」	「很有可能出现相同的字符，但是他们这个指针不一样」；绝对值最大取到 `n`，开 `n` 个下标就溢出；命名要用 `p`、`q`（「两条到底谁长谁短你提前不知道」）；`>=` 会比 `>` 多走一步	`map` 键「一定要定义成这个 `LNode *`」；数组开 `n+1` 个并显式清零（⚠️ 清零那句老师没念）；用具体例子（7 和 5）验边界	`[01:04:59]` `[01:19:59]` `[00:40:52]` `[00:43:01]`

7 考场速查卡

```
【零】写代码前先确认三件事
  ① 带不带头结点？→ 链表大题默认「带」
  ② 题干的域名与类型是什么？→ 照抄（第 18 题是 char，不是 int）

【一】结点定义（必须默写，末尾分号不能丢）
typedef struct LNode {
    int data;                 // 题目是 char 就写 char
    struct LNode *link;       // 题目叫 next 就写 next
} LNode, *LinkList;

【二】求表长（都极其高频；头结点不计入长度）
写法一：cur = list->next;  while (cur != NULL)      { x++; cur = cur->next; }
写法二：cur = list;        while (cur->next != NULL) { x++; cur = cur->next; }
```

```
【三】快慢指针三件套
① 倒数第 k 个：fast 先走 k 步（★ 带 fast != NULL）→ 一起走
               → if (slow == list) 说明不存在
② 找公共结点：求两条表长 → 长的先走长度差步（★ 严格大于，不是大于等于）
               → while (p != q) → 返回 p（无交点时两者同时为 NULL）
③ 找中点：    while (fast != NULL && fast->next != NULL)   // ★ 两个条件
                   { fast = fast->next->next; slow = slow->next; }
              // 奇数个 → 正中间；偶数个 → 前半段最后一个
```

```
【四】链表反转（头插法）
cur = head->next;
head->next = NULL;                  // ★ 第二句最容易漏
while (cur != NULL) {
    temp = cur->next;               // ① 存后继
    cur->next = head->next;         // ② 接上已反转串
    head->next = cur;               // ③ 挂回头结点后面
    cur = temp;                     // ④ 推进
}

【五】删除（第 19 题）—— cur 从头结点出发，判的是 cur->link
   出现过 → cur->link = cur->link->link;   // ★ cur 不动！
   没出现 → a[v] = 1;  cur = cur->link;    // 只有这一步才后移

【六】重排链表（第 20 题）= ③ + ④ + 拆链 + 交替合并
1) 找中点 slow
2) slow 当「头结点」，用【四】反转 slow 后面那半段
3) q = slow->next;  slow->next = NULL;      // ★ 先取后断，顺序不能反
4) p = L->next;  cur = L;
   while (q != NULL) {                      // ★ 以短的那半段为条件
       cur->next = p; p = p->next; cur = cur->next;
       cur->next = q; q = q->next; cur = cur->next;
   }
   cur->next = p;                           // ★ 奇数个时补最后一个，不能漏

【七】复杂度：时间 O(n)（两条链表 O(m+n)）；没开额外空间 O(1)，
      开了辅助数组 / map 则 O(n)。求表长的开销也要算进时间复杂度里。

【八】得分策略（老师口径）：满分 15 分。思路对 + 代码写出来 ≈ 10 分。
      结点定义单独有分 —— 不会做也要先写上去。
      实在没思路 → 数组兜底，「随便写都比（空着）强」。
```

如果只记五件事

① **结点定义照抄题干**（域名、类型都不能改），末尾分号不能丢；

② **两个「与运算」**：快指针先走 k 步要带 `fast != NULL`；快慢指针求中点的循环条件要写两个；

③ **删除必须停在前一个结点，且删完 `cur` 不动**；

④ **反转 / 拆链都是「先把要用的结点拿在手里，再动指针」**——`temp` 先存后继、`q = slow->next` 先取后断；

⑤ **第 20 题三步**：找中点 → 反转后半段 → 交替合并（合完记得补一句 `cur->next = p;`）。

附：本资料的来源与边界

来源	内容	可靠性与边界
B 站《408 各科大题知识点强化课》第 2 讲「数据结构 · 链表算法题」分 P4，音频 **2 小时 23 分 55 秒**（8635.87 s）	§1–§7 全部（除标注为「本资料补」的内容）	本地 Whisper `small` 模型 CPU 转写，讲稿 **1694 行**，覆盖 `[00:00:24]`–`[02:22:58]`，即**全程**；错字密度高，引用处已校正并注明。**全程没有板书**，所有代码都是按他逐句口述的命令顺序还原的。音频中约 29 分钟是休息（见 §1），不是转写缺失。
公开真题资料；教材 / 语言通识；分 P3	§1.2 年份认定、§5.1.1 题面；§2.2 末尾的语法规范、§5.3.2 的数组清零行；§3.4 数组兜底法的来源	真题年份**不是老师讲的**——老师只对第 20 题说出年份（「19 年」），其余按题面比对认定，依据见 §1.2，**请以真题原卷为准**。教材通识部分已在正文逐处标注「本资料补」；分 P3 已单独成篇，本资料只引用其结论。

> ⚠ **转写含糊、按上下文推断的地方（正文已分别标注）**
>
> ① `[00:20:54]` **第 17 题写法二的形式**：据「先用再减」「并不满足小于等于 0」还原为 `if (k-- <= 0)`；他**没有逐字念出循环体**。
>
> ② `[00:19:13]` **第 17 题数组法复杂度**：转写「O1 和 O1」无法直译，按上下文推断为 O(n) / O(1)。
>
> ③ `[00:43:25]` **第 18 题的表名与循环形式**：链表名转写为「str1 / STR2」，推断为 `str1` / `str2`；长度比较那句转写「大雨等于那 2」无法直译，按他举的 7/5 例还原成 `while (len1 > len2)`——**写成 `for` 也完全可能**。

> ⚠ **转写含糊、按上下文推断的地方（续）**
>
> ④ `[01:09:00]`–`[01:25:51]` **第 19 题的样例与实现**：原始序列在转写里前后不一致（「复制 5」「附期」无法直译），**正文只保留可确证的「结果是 `21, -15, -7`」**；「题干给出 |data| 小于 n」那句转写为「小元的」，按上下文推断；取绝对值**没念**是 `abs()` 还是手写 `if`；辅助数组初值他**假定为 0**，初始化那一句讲稿里没有。
>
> ⑤ `[01:50:15]`–`[02:05:58]` **第 20 题的若干处**：函数名「起一个函数叫 FUNC」其实没给出；结点定义没有逐字念出（拼写请对原题）；`p` 的口述前后矛盾，按上下文还原为 `p = L->next;`；「同一句话出现了两次」指 `slow->next = NULL;` 在反转段和拆链段各一次；图例数字听错得厉害（「1 17 26 17 26 35 4」），按他后面「你看 172635 还缺一个 4」还原为 `1…7` → `1,7,2,6,3,5,4`。
>
> ⑥ `[00:03:53]`、`[01:47:37]` **两处口径还原**：`LinkList` 别名按分 P3 的讲法还原；快慢指针起点按他「都从头结点开始」写。**题号全部是他题单上的编号**（非真题第 3 题、课后题第 15 题、17–20）；本讲没有提到任何教材书名与页码。

老师没有讲 / 本讲没有覆盖的内容

① **第 17 题的前两种方法、第 19 题的数组法都只有思路、没写代码**；判断是否有环、判断回文链表、第 20 题的合并段这三道课后题同样只给了思路。

② **双链表的代码细节没有讲**（分 P3 说过「后面写代码还是有一些东西要注意的」）；**链表插入的「先接后断」也没有讲。**

③ **树、图的代码题明确延后**——「下次上课我们就不讲代码题了」（`[02:22:34]`）。本讲**没有引官方答案**；回放与笔记在网盘，本资料无此材料，无法交叉核对。

整合自 2 小时 23 分 55 秒网课讲稿（本地 Whisper 转写、无板书，代码按口述逐句还原）+ 公开真题资料的年份比对 · 未上传任何材料到第三方服务 · 生成于 2026-10-10